HW2

- 姓名：陈雨萱
- 学号：23300240034
- 专业：计算机科学与技术（拔尖人才计划）

关键技术实现

setnamemaps.cc

该文件完成了符号表的构建，主要任务是遍历AST并收集所有声明信息，建立名字映射。它完成的所有功能如下表所示：

- 收集类信息
  - 记录所有定义的类名，其中main作为特殊类__$main__处理；
  - 记录类之间的继承关系；
- 收集方法信息
  - 记录每个类中的方法名；
  - 记录方法的形参列表和返回类型，返回类型使用 _return_方法名来特殊标记;
- 收集变量信息
  - 类变量：记录在 classVar 映射中
  - 局部变量：记录在 methodVar 映射中
  - 形参：记录在 methodFormal 映射中
- 重复声明检查
  - 同一类中不能有同名类变量，同一方法中不能有同名局部变量，同一方法中不能有同名形参
  - 允许局部变量和形参同名，也允许子类变量隐藏父类变量
- 错误处理
  - 发现重复声明立即报错并退出，然后输出错误的具体内容与位置信息

采用了Visitor Pattern，为每种AST节点类型提供对应的visit方法，从而将符号表构建的逻辑和AST节点类分离，在添加新的遍历逻辑时不需要修改AST节点类。 用name_maps来管理符号表，存储各种标识符信息。在遍历过程中，通过current_visiting_class/method来维护当前访问的类和方法上下文，从而正确地将声明归类到对应的作用域，这里可以区分不同作用域里面的同名变量。

对于MainMethod，我们将它处理为一个特殊的类"**$main**"，并且它对应的返回类型是"__return_main"，这里注意我们不在意MainMethod是不是有返回类型，如果有，必须是int（在语义分析阶段）。对于返回类型，我们将它处理为一个特殊的参数类型，然后放在参数的末尾加入形参列表。

非常重要的是，在符号表构建阶段，VarDecl的Visitor中需要做一项错误检查：即对重复声明的检查。因为在语义分析阶段，符号表中已经有这些变量了，再做重复检查会对每个变量都报错（当然硬要处理不是不行，但是麻烦）。为了方便，可以在符号表构建阶段就发现这种重复声明的错误，如果出现变量/参数的重复声明，在符号表构建阶段就报错，输出信息并终止程序。具体操作是，在VarDecl的AST_Name_Map_visitor中，在添加变量之前，先检查当前作用域中是否已经存在了同名声明。这里需要考虑类变量、局部变量和方法参数。需要强调，方法的参数和它的变量是可以重复的，因为它们的使用语法不一样。此外，因为FDMJ有Variable Hidding，所以我们只考虑同一作用域。

semantanlyzer.cc

该文件将测试文件生成的AST文件作为输入，对它进行语义分析，主要做了类型检查、语义规则验证，并在原来的AST节点上添加语义信息，得到新的AST树。它完成的所有类型检查如下所示：

1. MainMethod：认为**不需要**要求main方法必须有return语句
2. ClassDecl：检查类的继承关系，要求只能单层继承。具体来讲，先看有没有父类，如果有，看父类是不是存在；检查类不能继承自身，父类不能有父类，不能间接造成循环继承。
3. VarDecl：检查类类型是否存在，不能未声明就被使用；检查初始化的类型是否匹配，不能把一个INT初始化給INTARRAY。注意，我们不能在语义分析阶段检查未声明的变量是否可以使用，即不检查变量是否指向一个具体的对象，这其实是不可判定的。
4. MethodDecl：FDMJ是允许Method Overriding的，但是要求子类与父类的参数个数和类型一致，并且返回类型需要满足，子类重写方法时返回的类型与父类方法返回类型一致或是它的子类。所以这里要看是否重载了父类方法，如果重载，对比形参列表和返回类型是否匹配。
5. If：检查条件表达式是否存在并为INT类型；它的语义信息是否存在；then分支是否存在；else分支如果存在，继续递归地分析。
6. While：检查while语句必须有
7. Assign：检查赋值语句的左值是否一定可赋值，即lvalue（变量，数组元素，对象字段）；检查左边和右边是否都有完整的语义信息；检查左右两边的基本类型是否相同，不能把ARRAY赋值给INT类型；对于类类型，只允许相同类赋值和子类赋值给类型匹配的父类，其他都不允许；对子表达式递归分析。
8. CallStm：检查方法调用是否有对象表达式并且有正确的语义信息；检查方法调用的对象是不是类类型；方法是否在当前类或者父类中存在；检查实参个数是否等于形参个数以及类型是否对应。
9. Continue & Break：检查了一下continue和break语句是否在循环内
10. Return：检查返回值的语义信息和期待的返回类型是否匹配，特别的，main要返回INT（但可以不返回）。同时也允许返回的是子类对象，其他情况都报错。
11. PutInt & PutCh & PutArray：检查要输出的是不是对应的类型。
12. ArrayExp：数组表达式要有正确语义信息，并且[]只能对数组使用，数组索引类型必须是INT，如果索引是常量整数则不能为负数。
13. CallExp：调用的对象和语义信息存在，并且对象只能是CLASS；检查方法是否在当前类或者父类中存在；检查实参个数是否等于形参个数以及类型是否对应；以及返回值必须要有并且不是左值；
14. ClassVar：对象必须是类并且有语义信息，类名必须存在或者在父类中存在；

15. Length：有语义信息并且参数必须是数组类型。
16. NewArray：有语义信息并且数组大小必须是INT。
17. NewObject：new表达式必须要有类名，并且类必须存在。
18. GetArray：要有语义信息，并且getarray()只能用于数组类型。
19. IdExp：这里其实不处理没有new就访问或者没有声明就访问的情况，而是见到了就给他一个默认的INT语义信息，不报错。
20. Type：不能出现FDMJ允许的三种之外的类型。

在语义分析过程中，我的设计是遇到第一个错误后不退出，而是将mode赋值为-1，直到检查并输出所有错误后，如果mode为-1就输出语义分析失败的提示信息，然后不生成对应的加了语义信息的AST树的XML文件。此外，我在加入语义信息过程中，将能加的语义信息都加了。

开发过程 & 测试结果

其实我只在结束的时候add并commit了一次，做的时候忘了，下次一定养成良好习惯：

Graph	Description	Date	Author	Commit
	master finish	16 Mar 2026 23:13	23300240034-chen...	73421e8c
	origin/HEAD origin/master added some more HW2 newtests	13 Mar 2026 14:15	xywang2772	655dc43c
	HW2 add newtest13,14,15,16	13 Mar 2026 12:08	xywang2772	c0befe0b
	Updated parser-linux-x86-64	12 Mar 2026 21:27	xseanwang	86923909
	Debugged Parser and XML<->AST converters	12 Mar 2026 16:34	xywang2772	961503d0
	Add new test files for HW2	12 Mar 2026 16:17	xywang2772	c49cbf69
	Merge branch 'master' of https://gitee.com/fudanCompiler/fducompilerh2026	11 Mar 2026 19:33	xseanwang	4f5628f3
	parser-linux-x86-64 added line info into 2.ast	11 Mar 2026 19:33	xseanwang	4ef363c1
	Update parser	10 Mar 2026 23:42	xywang2772	4bbf5450
	Updated Parser	10 Mar 2026 22:44	xywang2772	74b49992
	Added location info to ast files	10 Mar 2026 21:21	xywang2772	d1c6939e
	Add linux-x86-64 parser	8 Mar 2026 17:36	xseanwang	b5f20b98
	HW2 added	7 Mar 2026 00:05	王晓阳	f366f01e
	Slight modification	26 Feb 2026 15:36	王晓阳	55136e8a
	HW1 published	23 Feb 2026 00:20	王晓阳	30b4d729
	OK	22 Feb 2026 21:32	王晓阳	71e07708
	Remove README.md	22 Feb 2026 21:31	王晓阳	27abdb32
	Initialization	22 Feb 2026 21:30	王晓阳	392043f5
	Initial commit	22 Feb 2026 21:28	王晓阳	a8569e5c

用这个表达一下我确实做了好几天吧：

C++ seman... 9+, M
C++ setnamem... 6
util
大纲
时间线 semantanlyzer.cc
已保存文件 4 小时
已保存文件 7 小时
已保存文件
已保存文件 7 小时
已保存文件
已保存文件
finish 23300... 19 小时
已保存文件 1 天
已保存文件
已保存文件
已保存文件
已保存文件
已保存文件
已保存文件
已保存文件
已保存文件
已保存文件
已保存文件
已保存文件
已保存文件
已保存文件
已保存文件
已保存文件
已保存文件 2 天
撤消/重做
已保存文件

测试结果：对所有测试文件，比对老师给的示例终端输出和我得到的终端输出，报错信息格式有差异，但是具体错误内容一致，没有缺少。另外我得到的加了语义分析的AST树与老师给的示例进行对比，也没有缺少任何语义信息，但是增加了一些，比如常数的语义信息（其实这个可加可不加，毕竟肯定是INT，肯定不是lvalue）。

参考资料

docs中的四个md文档，ChatGPT（生成符号表过程中如何处理main和return）和DeepSeek(检查完善我在语义分析阶段还有哪些可以做的类型、错误检查我没有做)。